

实验报告

实验 4 运算部件设计

课程名称

数字逻辑与计算机组成

指导教师

吴海军

1 实验目的

- 掌握先行进位部件 CLU 和先行进位加法器 CLA 的设计方法。
- 掌握 32 位快速加法器和标志位的设计方法。
- 掌握 32 位桶形移位器的设计方法。
- 掌握 RV32I 算术逻辑部件的设计方法。

2 实验环境

- 平台：Logisim

3 实验内容

3.1 4 位先行进位加法器 CLA

3.1.1 实验整体方案设计

- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：

X,Y	输入数据
Z	输出数据
Cin	进位输入
Cout	进位输出
Pg	组间进位传递函数
Gg	组间进位生成函数

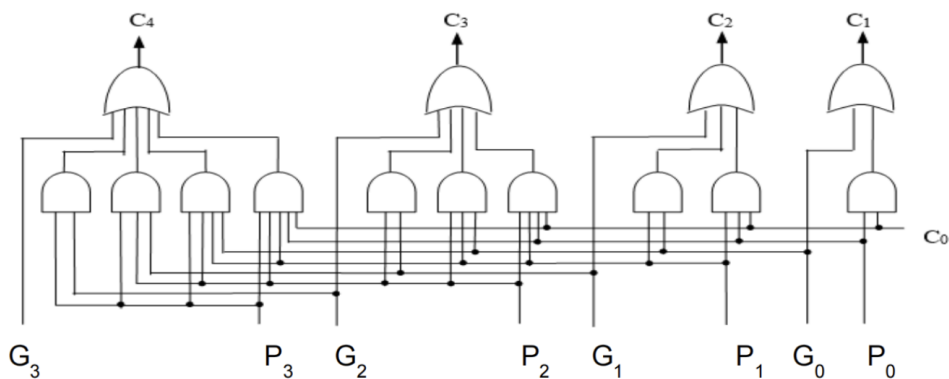
Table 1: CLA4 引脚作用

P1~P4	进位传递函数
G1~G4	进位生成函数
C0	进位输入
C1~C4	进位输出
Pg	组间进位传递函数
Gg	组间进位生成函数

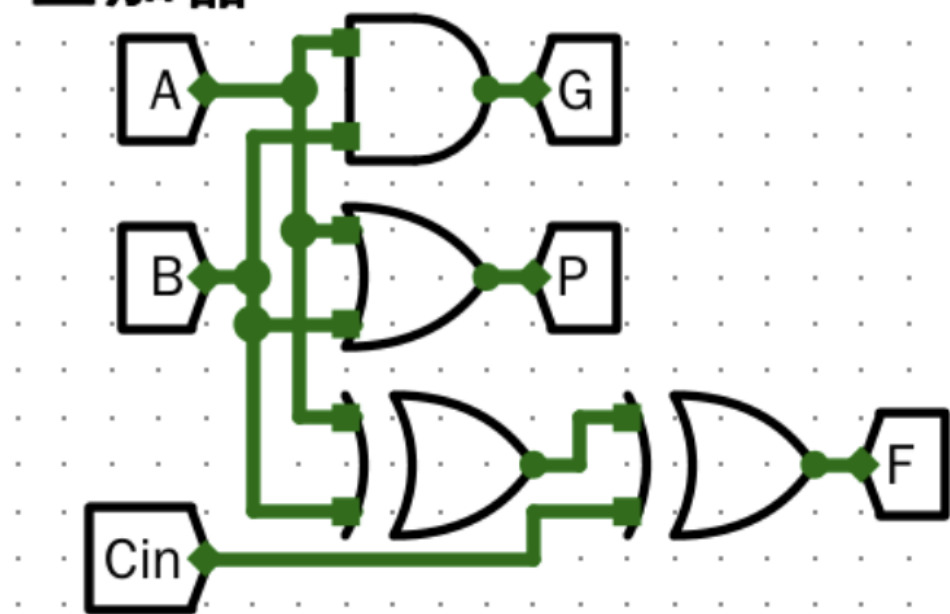
Table 2: CLU4 引脚作用

3.1.2 实验原理图和电路图

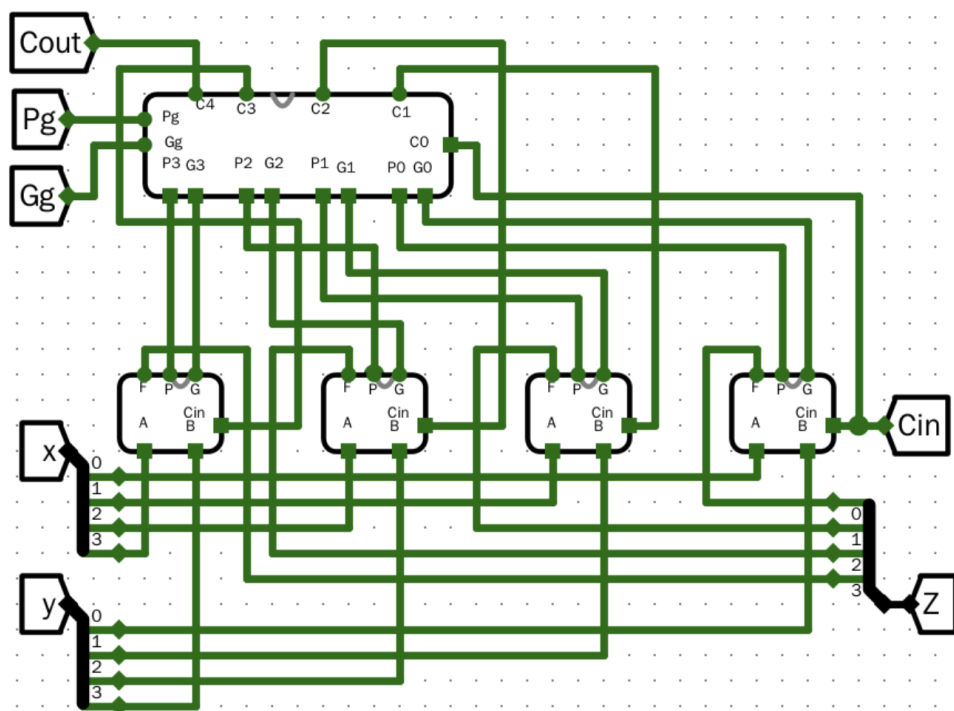
4 位先行进位部件原理图



全加器

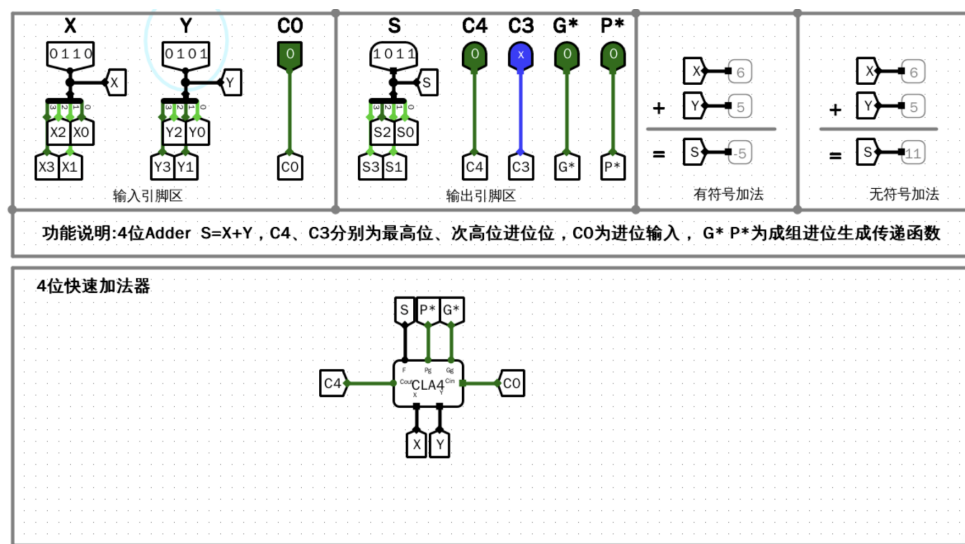


4 位快速加法器子电路图



3.1.3 实验数据仿真测试图

4 位快速加法器仿真测试图



3.1.4 错误现象及分析

本实验中没有发生错误。

3.2 16 位两级先行进位加法器实验

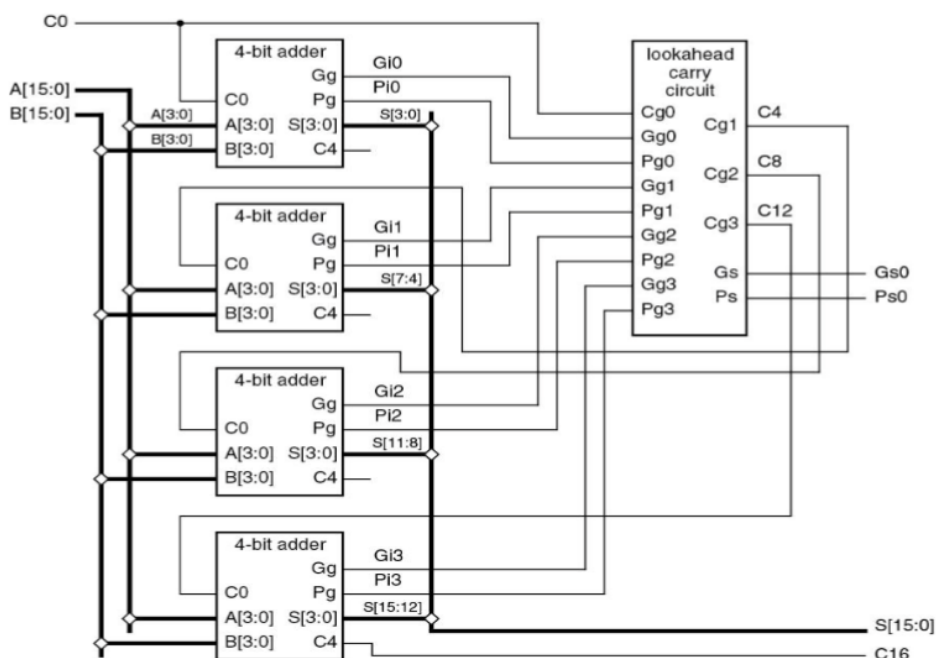
3.2.1 实验整体方案设计

- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：

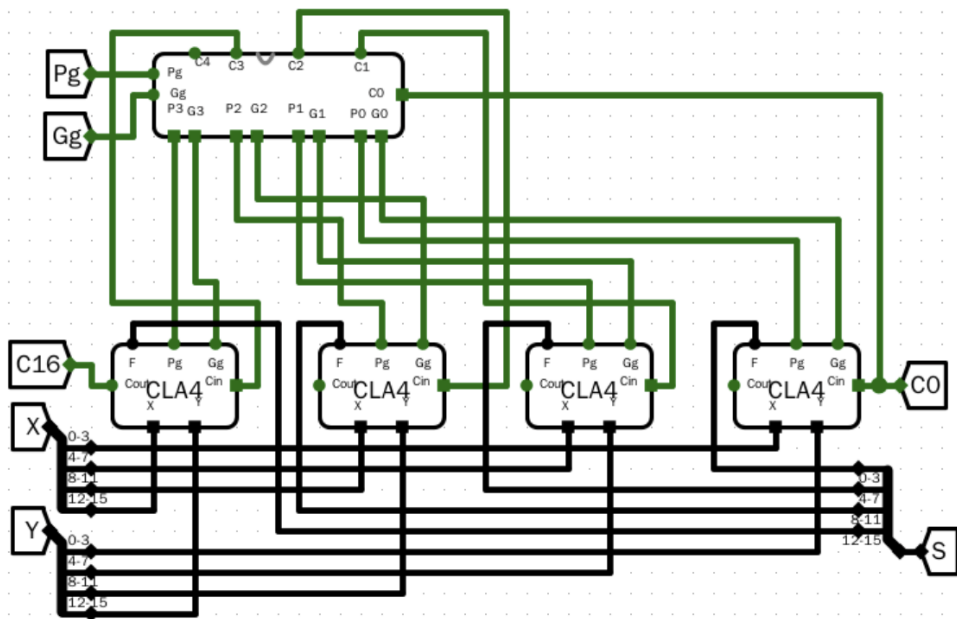
X,Y	输入数据
S	输出数据
C0	进位输入
C16	进位输出
Pg	组间进位传递函数
Gg	组间进位生成函数

3.2.2 实验原理图和电路图

16 位两级先行进位加法器原理图

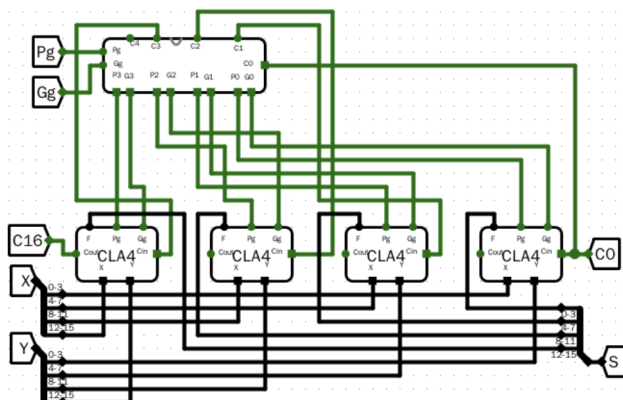
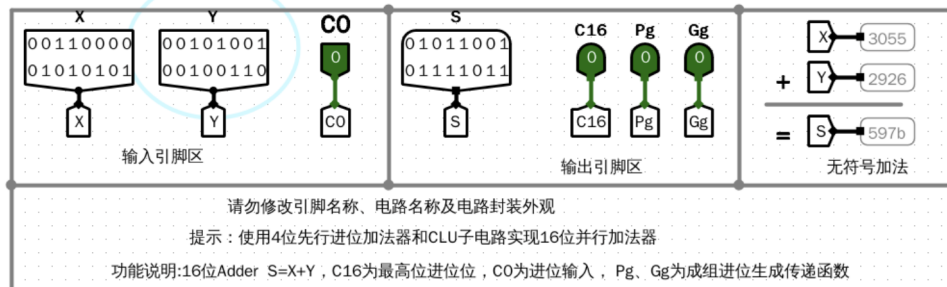


16 位两级先行进位加法器电路图



3.2.3 实验数据仿真测试图

16 位两级先行进位加法器仿真测试图



3.2.4 错误现象及分析

本实验中没有发生错误。

3.3 32 位快速加法器实验

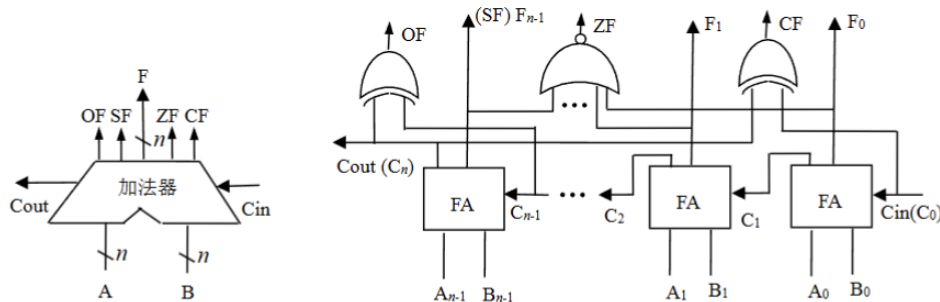
3.3.1 实验整体方案设计

- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：

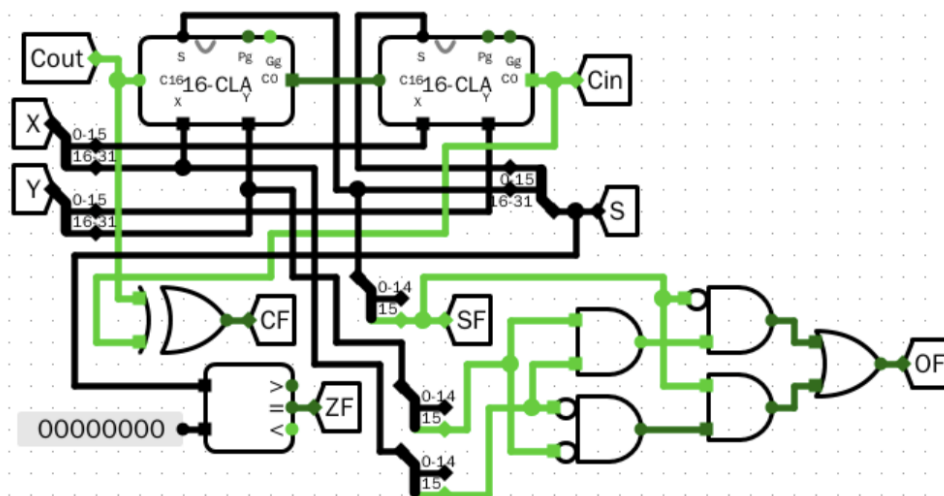
X/Y	输入数据
S	输出数据
Cin	进位输入
Cout	进位输出
CF、SF、OF、ZF	标志位

3.3.2 实验原理图和电路图

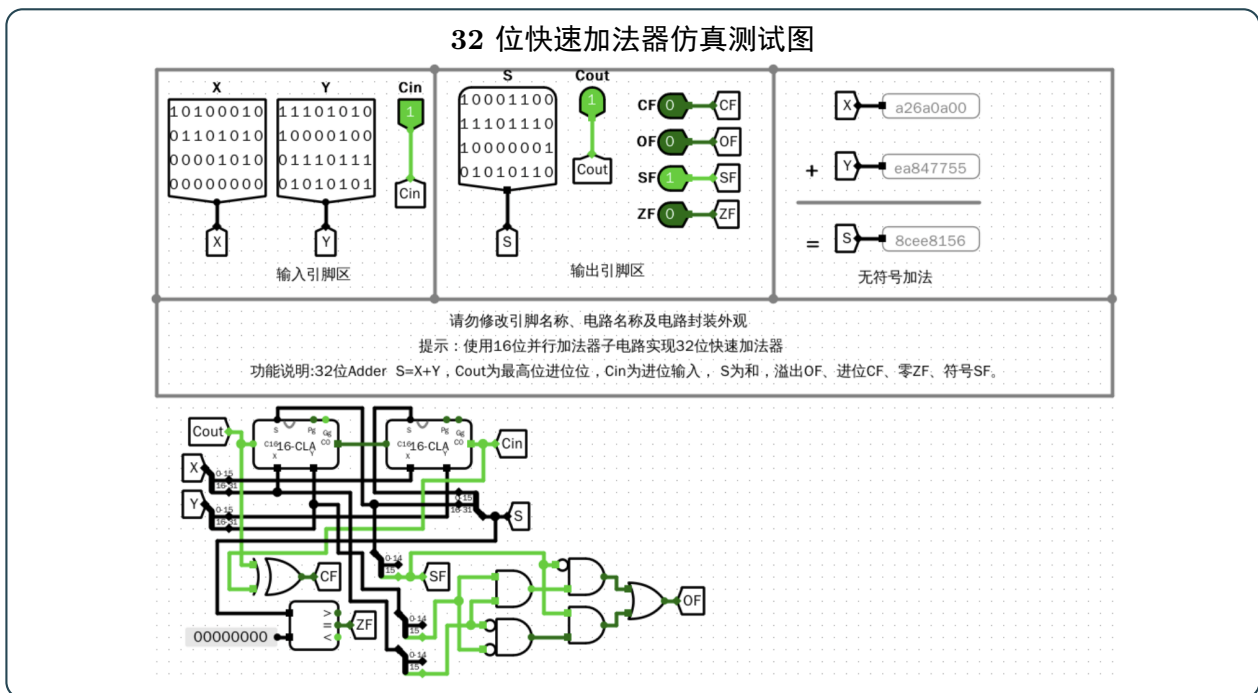
加法器标志位生成原理图



32 位快速加法器子电路图



3.3.3 实验数据仿真测试图



3.3.4 错误现象及分析

在 OF 标志位的组合逻辑电路设计中,错误的认为异或门可以代替功能,导致错我。修正为 $OF = X_{n-1} \cdot Y_{n-1} \cdot \overline{F_{n-1}} + \overline{X_{n-1}} \cdot \overline{Y_{n-1}} \cdot F_{n-1}$ 后电路正常工作, 成功通过了测试。

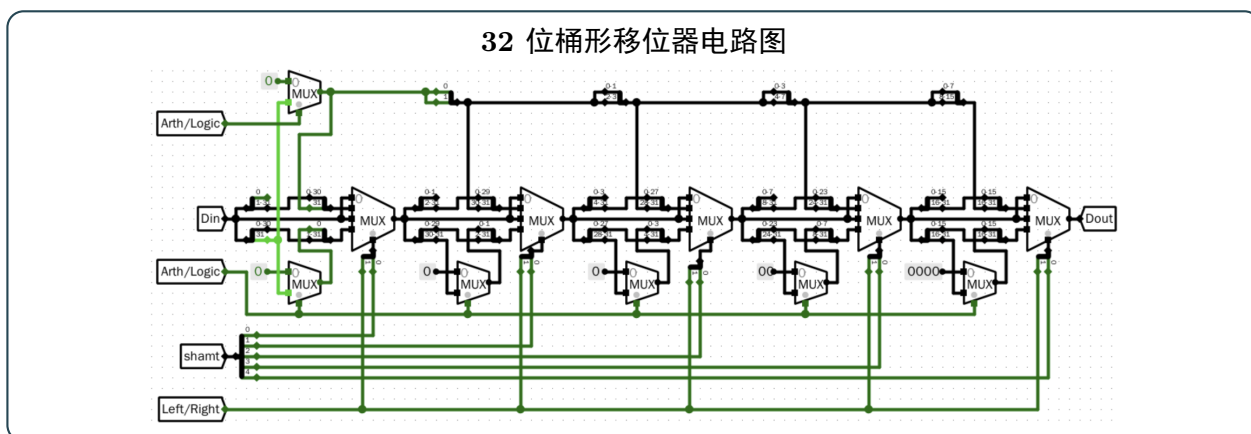
3.4 32 位桶形移位器实验

3.4.1 实验整体方案设计

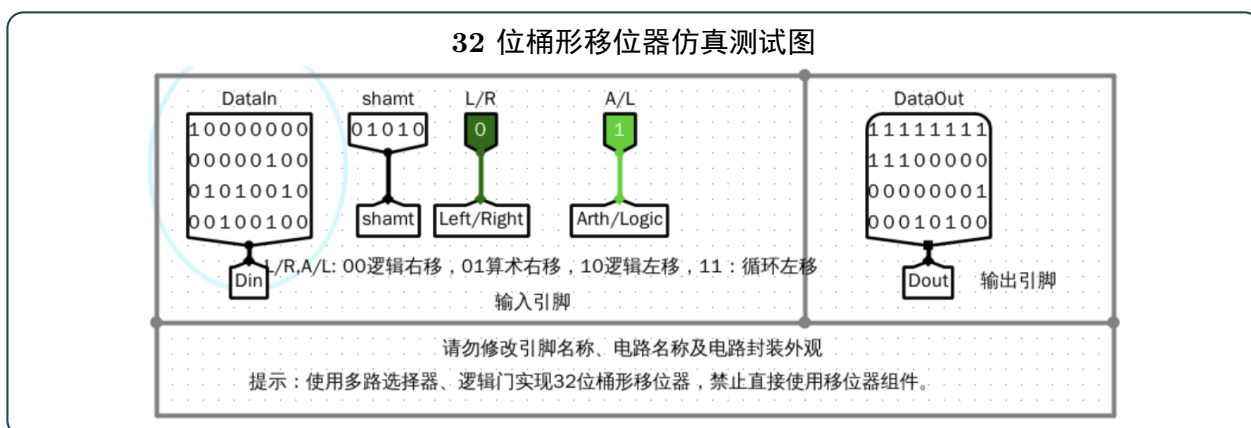
- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 功能表：

Din	shamt	LR	AL	Dout
x	$y(1 \leq y \leq 32, \text{下同})$	0	0	逻辑右移 y 位, 移入 0
x	y	0	1	算数右移 y 位, 移入符号位
x	y	1	0	逻辑左移 y 位, 移入 0
x	y	1	1	循环左移 y 位, 将移出位移入

3.4.2 实验原理图和电路图



3.4.3 实验数据仿真测试图



3.4.4 错误现象及分析

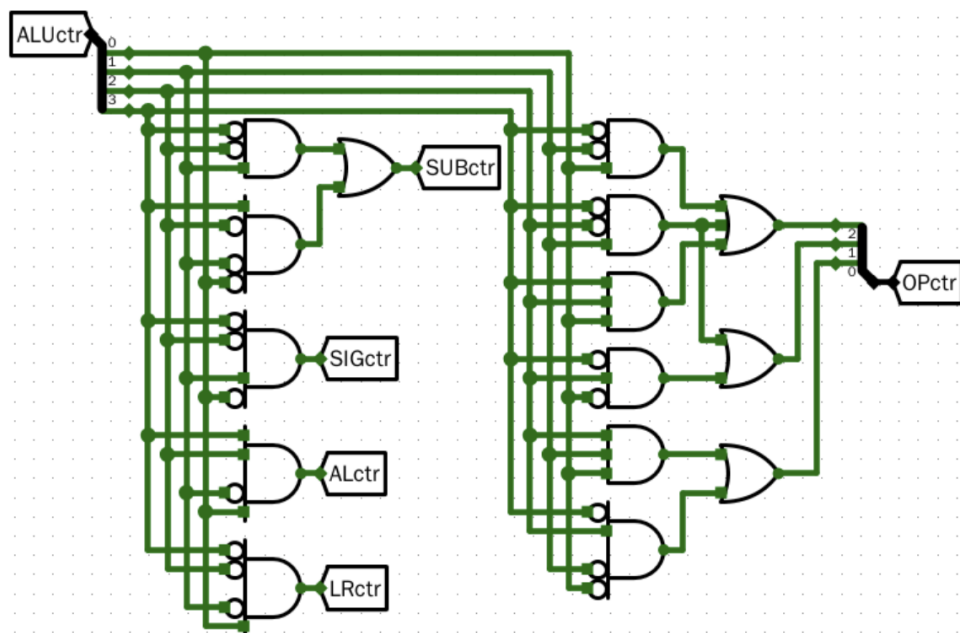
实验中没有发生错误。

3.5 RV32I 算术逻辑部件实验

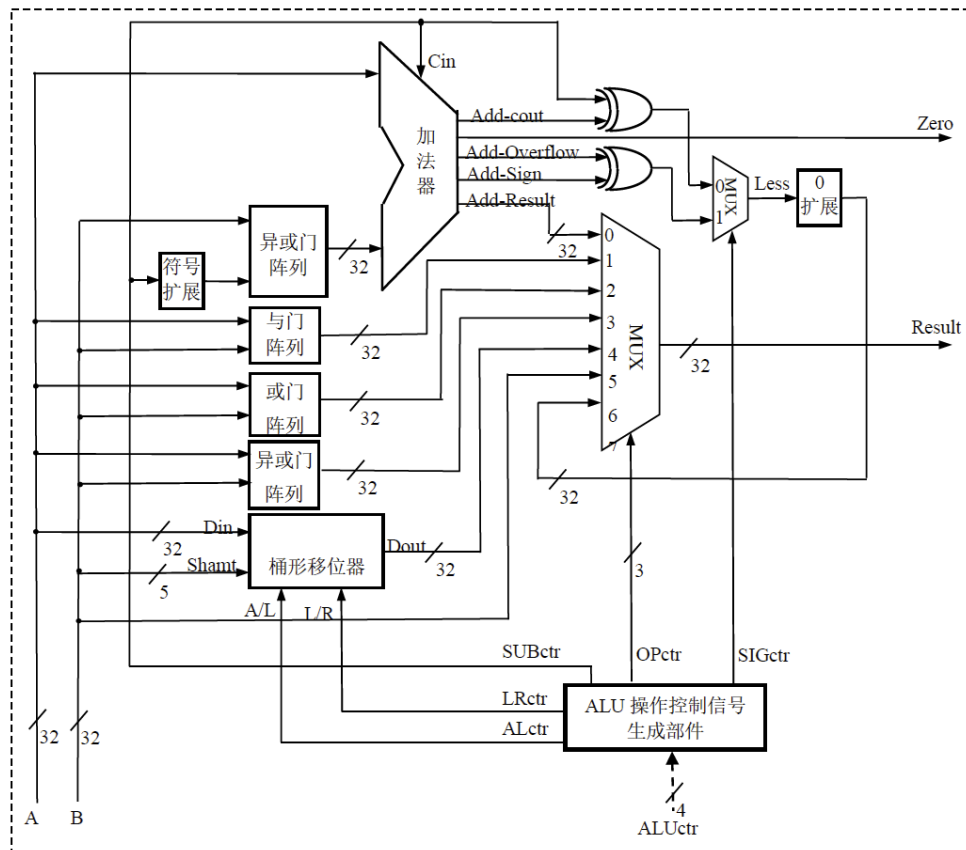
- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 功能表：

ALUctr	操作控制信号
SUBctr	SUBctr=1 时，ALU 作减法；SUBctr=0 时，做加法
SIGctr	SIGctr=1，执行“带符号整数比较小于置 1”；SIGctr=0，执行“无符号数比较小于置 1”
LRctr	控制桶形移位器移位方向
ALctr	ALctr=0 逻辑移位；ALctr=1，算术移位
OPctr	控制选择运算结果作为 Result 输出

ALUctr 操作控制部件电路图



ALU 组件原理图



3.5.1 功能测试

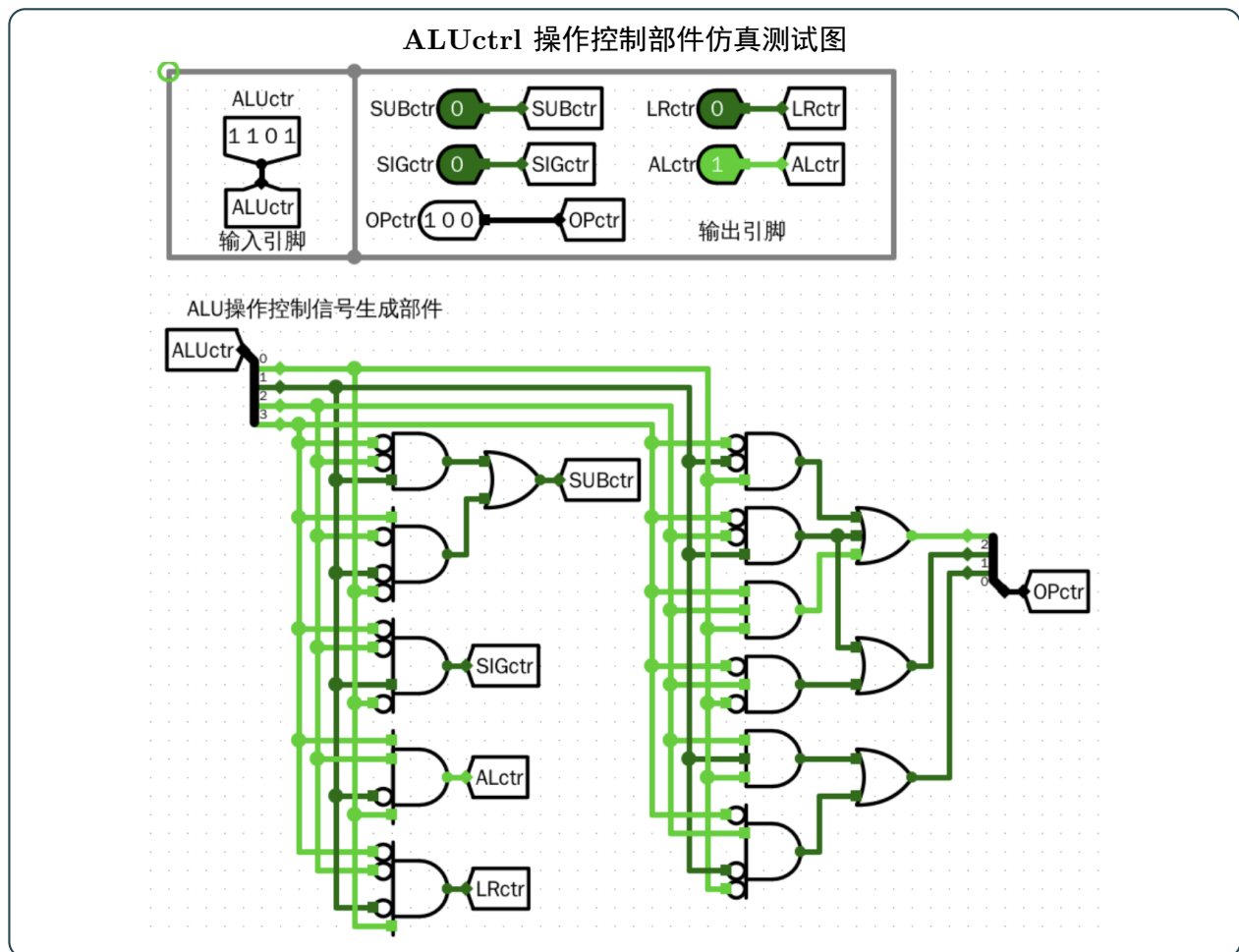
ALUctr <3:0>	指令操作类型	SUBctr	SIGctr	ALctr	LRctr	OPctr <2:0>	OPctr 的含义
0000	auipc, addi, add, jal, jalr, lb, lh, lw, lbu, lhu, sb, sh, sw: 加法	0	×	×	×	000	选择加法器的结果输出
0001	sll, slli: 逻辑左移	×	×	0	1	100	选择移位器结果输出
0010	slt, slti, beq, bne, blt, bge: 带符号数小于比较	1	1	×	×	110	选择小于置位结果输出
0011	sltu, sltui, bltu, bgeu: 无符号数小于比较	1	0	×	×	110	选择小于置位结果输出
0100	xor, xori: 异或	×	×			011	选择“按位异或”结果输出
0101	srl, srli: 逻辑右移	×	×	0	0	100	选择移位器结果输出
0110	or, ori: 或运算	×	×	×	×	010	选择“按位或”结果输出
0111	and, andi: 与运算	×	×	×	×	001	选择“按位与”结果输出
1000	sub: 减法	1	×	×	×	000	选择加法器的结果输出
1101	sra, srai: 算术右移	×	×	1	0	100	选择移位器结果输出
其余	(未用)						
1111	lui: 取操作数 B	×	×	×	×	101	选择操作数 B 直接输出

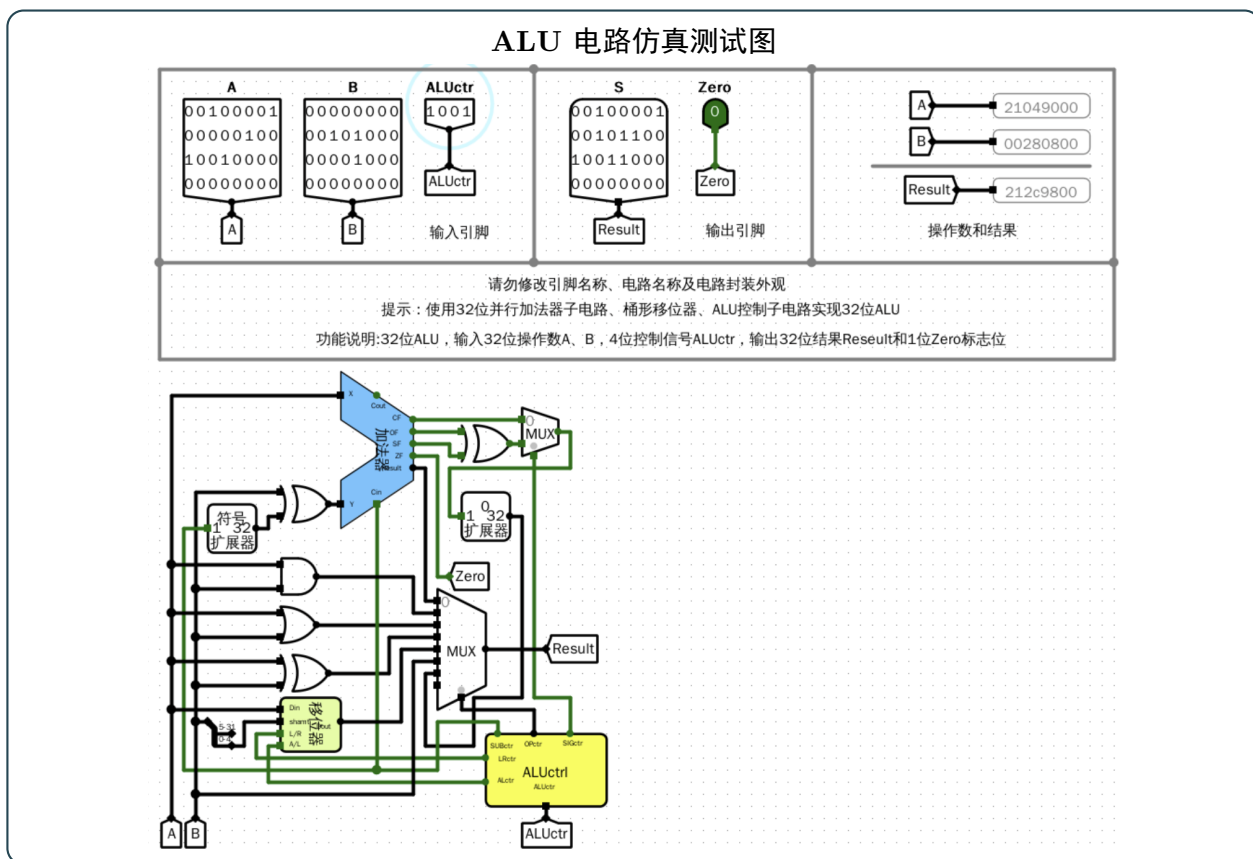
表 4.2 ALU 功能测试表

序号	A	B	ALUctr <3:0>	Result	Zero
1	0x80000000	0x7fffffff	0000	0xffffffff	0
2	0x01	0x7fffffff	0001	0x80000000	0
3	0x80000000	0x7fffffff	0010	0x00000001	0
4	0x80000000	0x7fffffff	0011	0x00000000	1
5	0x80000000	0x7fffffff	0100	0xffffffff	0
6	0x80000000	0x7fffffff	0101	0x00000001	0
7	0x80000000	0x7fffffff	0110	0xffffffff	0
8	0x80000000	0x7fffffff	0111	0x00000000	1
9	0x80000000	0x7fffffff	1000	0x00000001	0
10	0x80000000	0x7fffffff	1101	0xffffffff	0
11	0x80000000	0x7fffffff	1111	0x7fffffff	0
12	0x80000000	0x80000000	0000	0x00000000	1
13	0x80000000	0x80000000	0010	0x00000000	1

序号	A	B	ALUctr <3:0>	Result	Zero
14	0x80000000	0x80000000	0011	0x00000000	1
15	0x80000000	0x80000000	0100	0x00000000	1
16	0x80000000	0x80000000	0101	0x80000000	0
17	0x80000000	0x80000000	0110	0x80000000	0
18	0x80000000	0x80000000	0111	0x80000000	0
19	0x80000000	0x80000000	1000	0x00000000	1
20	0x80000000	0x80000000	1101	0x80000000	0
21	0x80000000	0x80000000	1111	0x80000000	0
22	0x5a5a5a5a	0xa5a5a5a5	0000	0xffffffff	0
23	0x5a5a5a5a	0xa5a5a5a5	0001	0x4b4b4b40	0
24	0x5a5a5a5a	0xa5a5a5a5	0010	0x00000000	1
25	0x5a5a5a5a	0xa5a5a5a5	0011	0x00000001	0
26	0x5a5a5a5a	0xa5a5a5a5	0100	0xffffffff	0
27	0x5a5a5a5a	0xa5a5a5a5	0101	0x02d2d2d2	0
28	0x5a5a5a5a	0xa5a5a5a5	0110	0xffffffff	0
29	0x5a5a5a5a	0xa5a5a5a5	0111	0x00000000	1
30	0x5a5a5a5a	0xa5a5a5a5	1000	0xb4b4b4b5	0
31	0x5a5a5a5a	0xa5a5a5a5	1101	0x02d2d2d2	0
32	0x5a5a5a5a	0xa5a5a5a5	1111	0xa5a5a5a5	0

3.5.2 实验数据仿真测试图





3.5.3 错误现象及分析

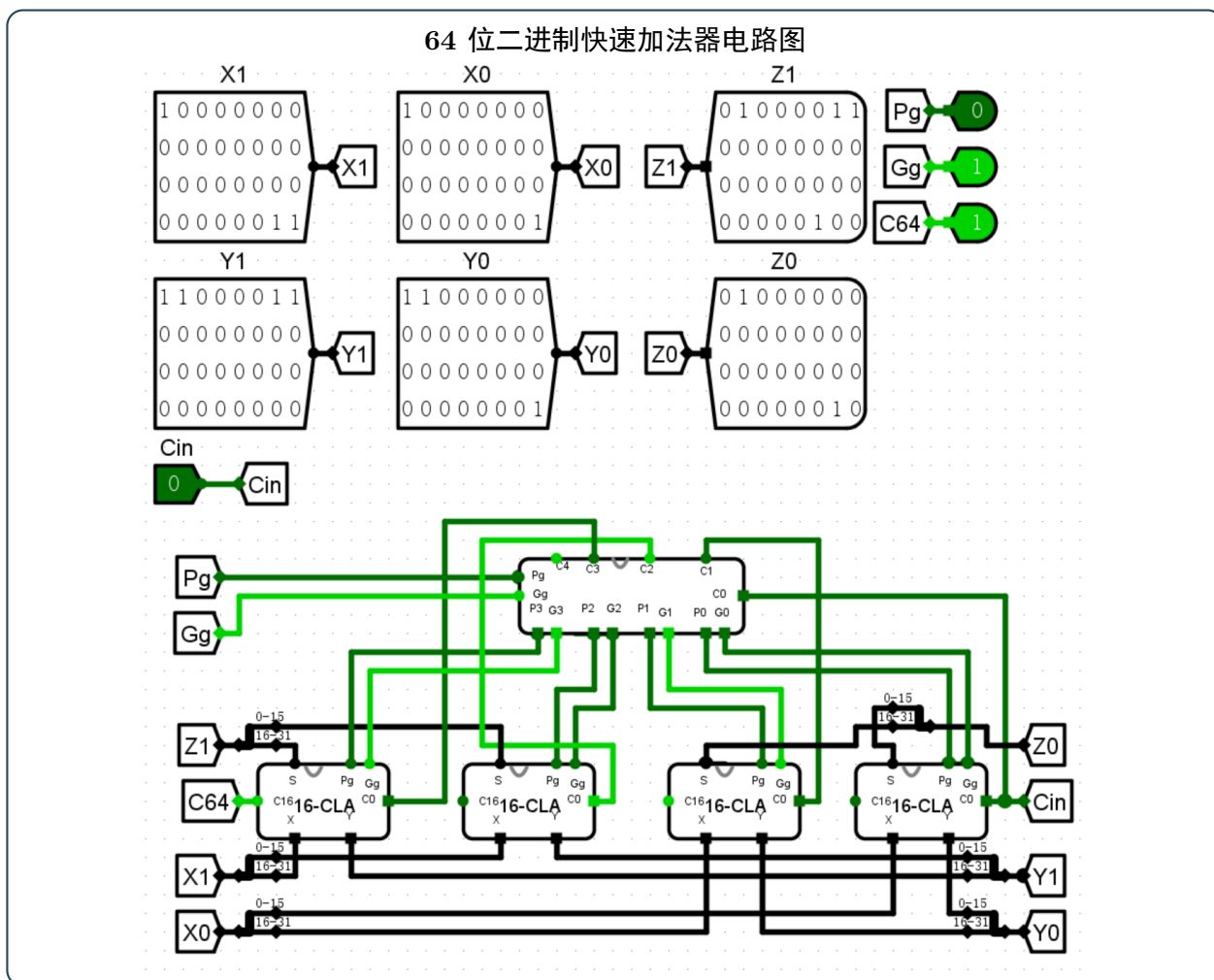
共修复了三处问题：

- 一个符号扩展器错误地使用了 0 扩展器，导致算术右移功能错误。
- L/R 与 A/L 控制信号连接反了，导致移位器功能错误。
- 两个针脚之间不小心链接。

4 思考题

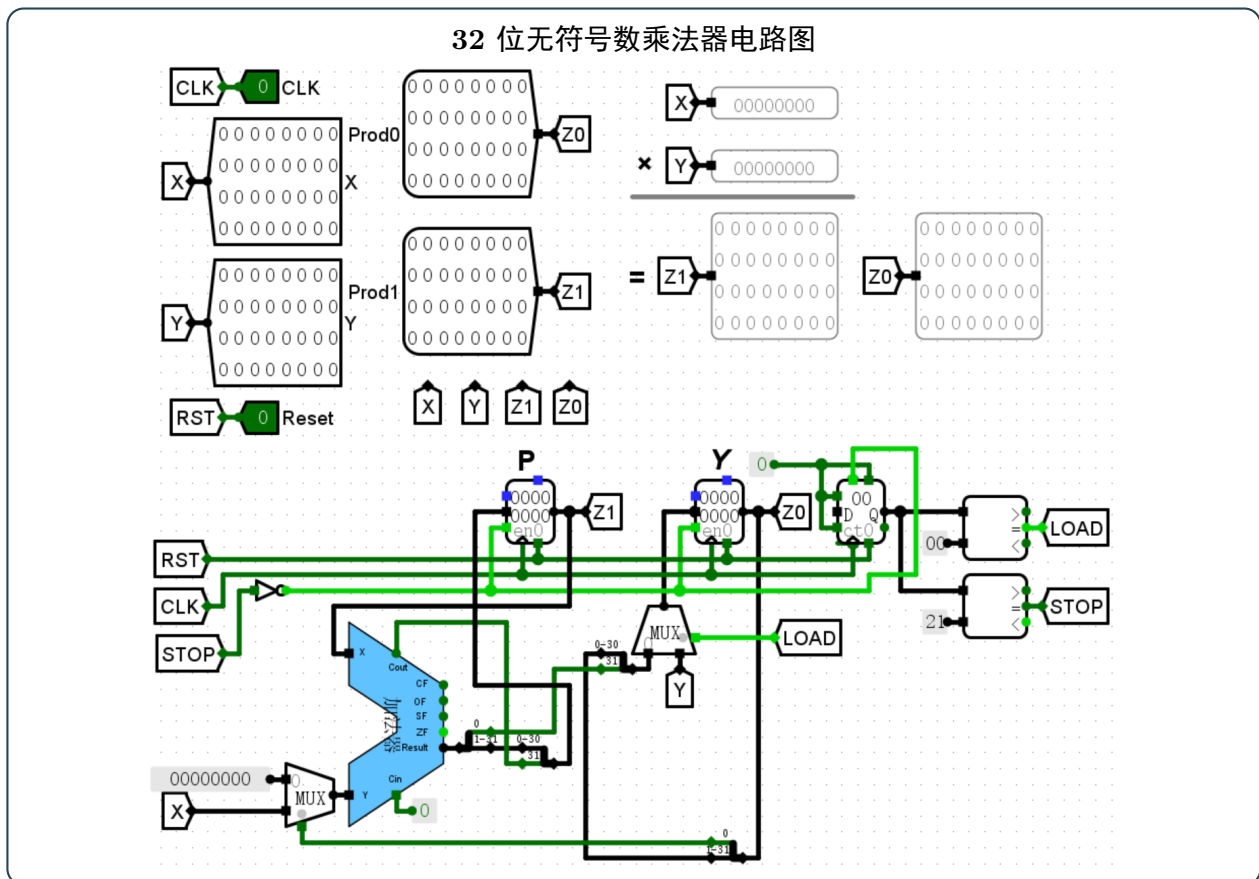
4.1 通过查找资料，实现一种 64 位二进制快速加法器的设计。

可以直接通过 16 位两级先行进位加法器的设计方法，将 64 位分为 4 个 16 位块，每个块使用一个 16 位两级先行进位加法器，并使用一个 CLU4 来连接这四个块。保存在文件 lab4.6.circ 中。



4.2 将实验 3 中的快速乘法器设计电路扩展到 32 位无符号数相乘，并探讨如何将该乘法器融合到实验中的 ALU 电路来实现乘法运算。

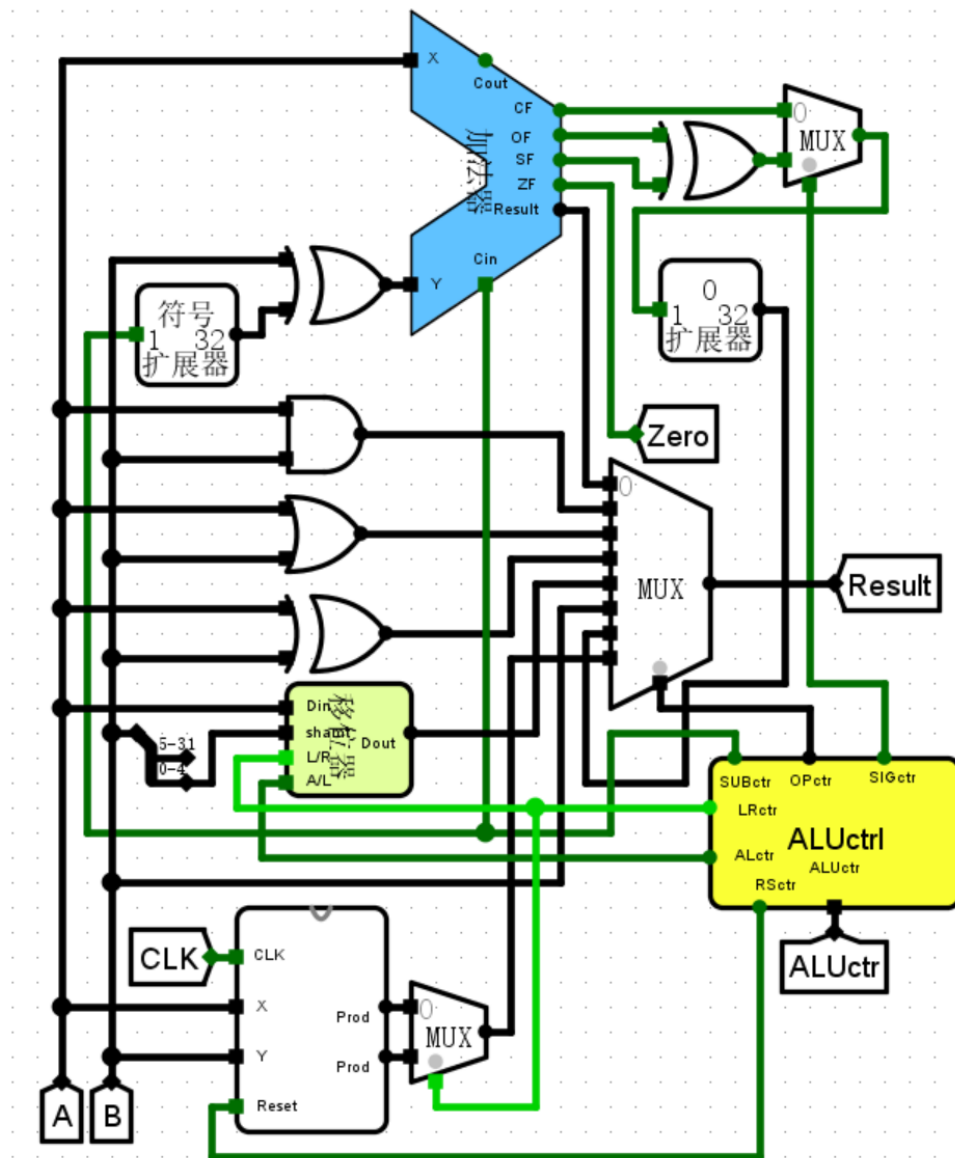
采用与实验 3.3 中相同的无符号乘法算法，使用线路进行位移而非桶形移位器。与 ALU 的融合将在下一题目中进行回答。保存在文件 lab4.7.circ 中。

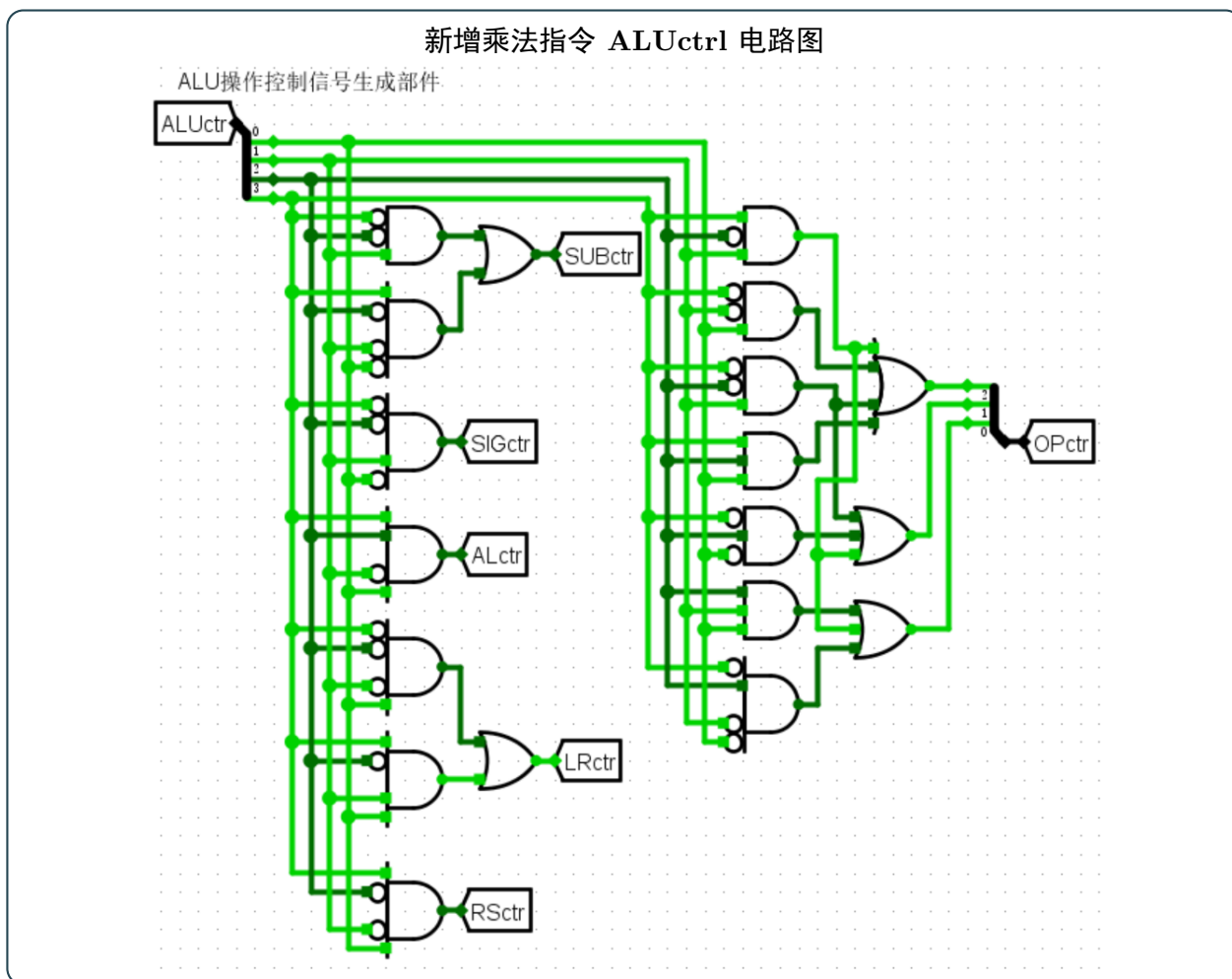


4.3 假设在 RV32I 中新增一条指令，导致在 ALU 中增加一个新运算操作，试修改 ALU 设计电路，并通过测试数据进行验证。

新增指令为乘法指令 mulr/mulh/mull, 分别对应应在 ALUctr=1001 时进行 reset, 在 ALUctr=1010 或 ALUctr=1011 时收取乘法运算的结果。由于输出结果为 64 位, 于是用两个不同的 ALUctr 值来选择输出低 32 位 (1010) 或高 32 位 (1011)。固定这两情况 OPctr 为 111, ALctr 分别为 0 与 1。而在 ALUctr=1001 时对输出没有要求, 添加一个 RSctr 引脚为 1。修改后的电路图如下所示。保存在文件 lab4.8.circ 中。

新增乘法指令 ALU 电路图





4.4 分析比较运算使用独立的比较器和使用减法运算通过标志位来实现两种方法的特性。

- 独立比较器的特点是针对比较运算专门设计，逻辑路径直接，输出结果明确，在只需要实现少量比较功能时较为简洁；但它需要额外的比较电路，硬件复用率较低，若同时支持有符号和无符号比较，还要分别处理不同的比较规则。
- 使用减法运算并结合标志位实现比较，则是先完成 $A - B$ ，再根据 ZF 、 SF 、 OF 、 CF 判断相等、小于或无符号小于等关系。其中，等于可由 ZF 判断，有符号小于可由 $SF \oplus OF$ 判断，无符号小于可由 CF 判断。这种方法可以复用加法器和标志位生成电路，面积更小，与 ALU 的加减运算统一性更好。